Rejection and Accept-Reject Sampling

Mark M. Fredrickson (mfredric@umich.edu)

Module 2: Generate

Computational Methods in Statistics and Data Science (DATASCI 406)

Rejection Sampling

Rejection Sampling

The idea of **rejection sampling** can be used when our **target RV** can be expressed as a subset of another **candidate RV**.

Examples:

- Conditioning on the variable itself, $X \sim \text{Poisson}(\lambda)$, $Y = X \mid X$ is odd
- Truncated distributions, e.g. $X \sim N(0, 1)$, $Y = X \mid a < X < b$
- Uniform points on the unit circle: $U_1, U_2 \sim U(-1, 1)$ (independent), $(V_1, V_2) = (U_1, U_2) \mid U_1^2 + U_2^2 \leq 1$

Procedure: generate from candidate, only keep results that meet criteria.

Example: Truncated Normal Distribution

A **truncated distribution** takes a given distribution and **limits the support**.

For example, $X \sim N(0, 1)$ and $Y = X \mid 0 \leq X \leq 1$. We can sample by drawing from X and only keeping those that fall in $(0, 1)$.

```
x <- rnorm(10000)
y <- keep(x, 0 < x & x < 1)
length(y) # number of samples kept
```

```
## [1] 3378
```

```
mean(y) # use the samples to estimate E(Y)
```

```
## [1] 0.4657
```

The truncated normal CDF

To check the sampler we need the distribution of Y. For $X \sim N(0,1)$, $Y = X \mid a \leq X \leq b$ and $a \leq y \leq b$,

$$F_Y(y) = P(X \leq y \mid a \leq X \leq b) = \frac{P(a \leq X \leq y)}{P(a \leq X \leq b)} = \frac{\Phi(y) - \Phi(a)}{\Phi(b) - \Phi(a)}$$

where Φ is the standard normal CDF (`pnorm`).

The denominator is the probability that a draw is kept:

```
pnorm(1) - pnorm(0)
```

```
## [1] 0.3413
```

So we throw away about two thirds of the draws from X.

Truncated normal quantiles

Set $F_Y(y) = p$ and solve for y:

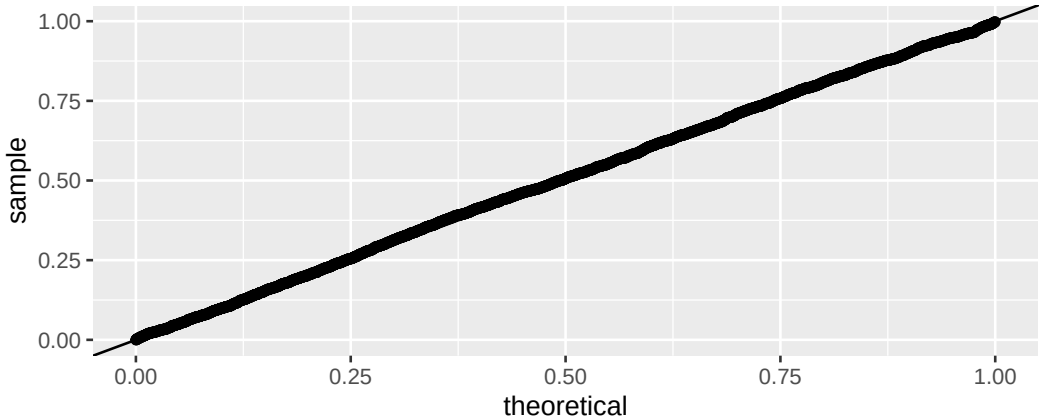
$$\Phi(y) = \Phi(a) + p\left[\Phi(b) - \Phi(a)\right] \Rightarrow Q_Y(p) = \Phi^{-1}(\Phi(a) + p\left[\Phi(b) - \Phi(a)\right])$$

The quantiles of Y come from the quantiles of the full normal (`qnorm`), evaluated at p rescaled into $[\Phi(a), \Phi(b)]$.

```
qtnorm <- function(p, a, b) {
  qnorm(pnorm(a) + p * (pnorm(b) - pnorm(a)))
}
```

Checking the rejection sampler

```
ggplot(data.frame(sample = y), aes(sample = sample)) +
  geom_qq(distribution = qtnorm, dparams = list(a = 0, b = 1)) +
  geom_abline(intercept = 0, slope = 1)
```



Rejection sampling summary

- The target is a candidate conditioned on an event: $Y = X \mid X \in A$.
- Draw from X and keep the draws that land in A.
- The fraction kept is $P(X \in A)$, which is $\Phi(b) - \Phi(a)$ for the truncated normal. When A is rare, most draws are wasted.
- Check the kept draws with a QQ plot against Q_Y.

Accept-Reject Method

Computing π

Consider the following method for approximating the value of π:

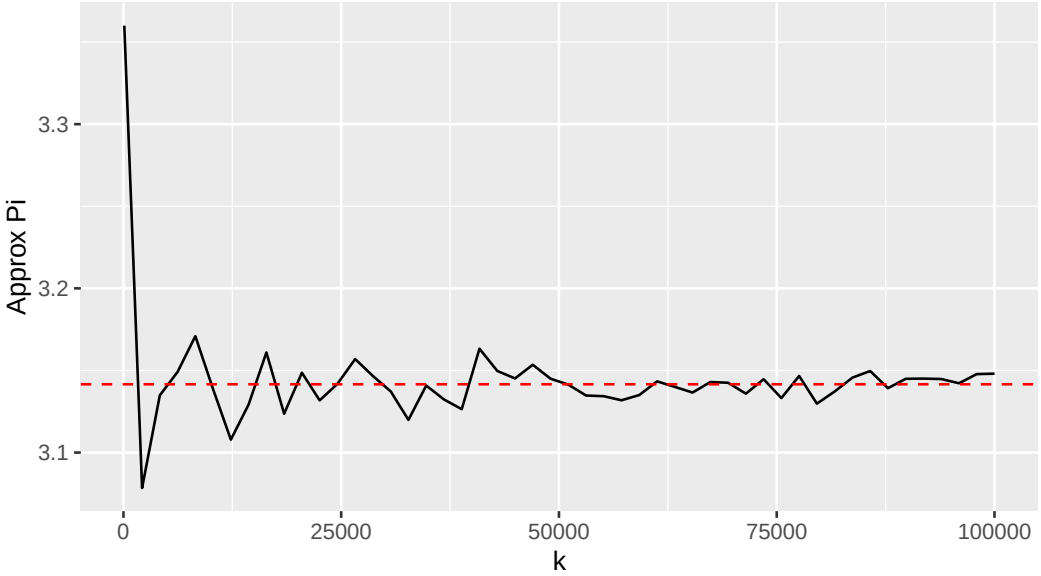
- Initialize $a = 0$
- Generate U_1 and U_2 (independent) from $U(0, 1)$
- If $\sqrt{U_1^2 + U_2^2} \leq 1$ then $a = a + 1$
- Repeat k times.
- Approximate π as

$$\pi = \frac{4a}{k}$$

Calculating in R

```r
approx_pi <- function(k) {
  u1 <- runif(k)
  u2 <- runif(k)
  sqs <- sqrt(u1^2 + u2^2)
  return(4 * sum(sqs <= 1) / k)
}
```

Approximation vs. k

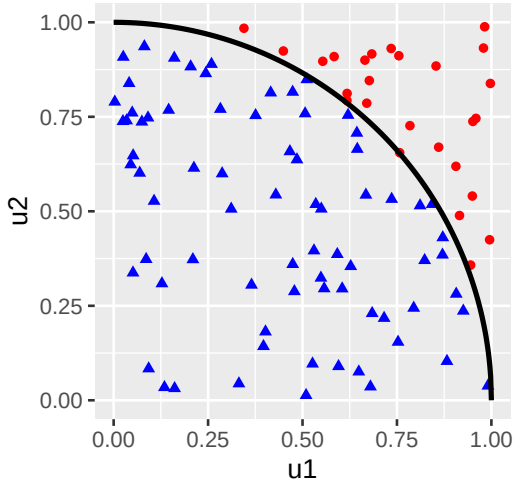


Why does it work?

Recall: the circle centered at $(0, 0)$ and with radius 1, has area π.

We "throw darts" at the upper quadrant of the circle and see how many hit the circle.

This proportion should be about $\pi/4$

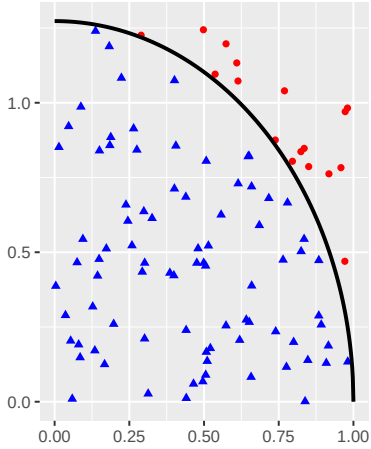

1.00
0.75
0.50
0.25
0.00
u2
0.00
0.25
0.50
0.75
1.00
u1

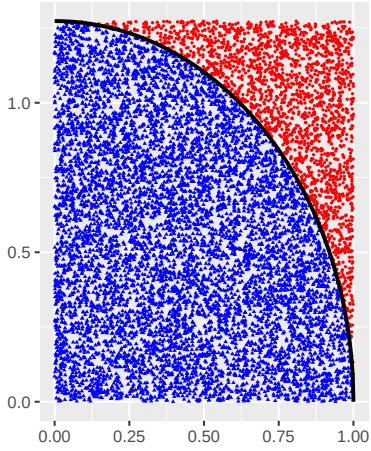
Another interpretation of Approximating π

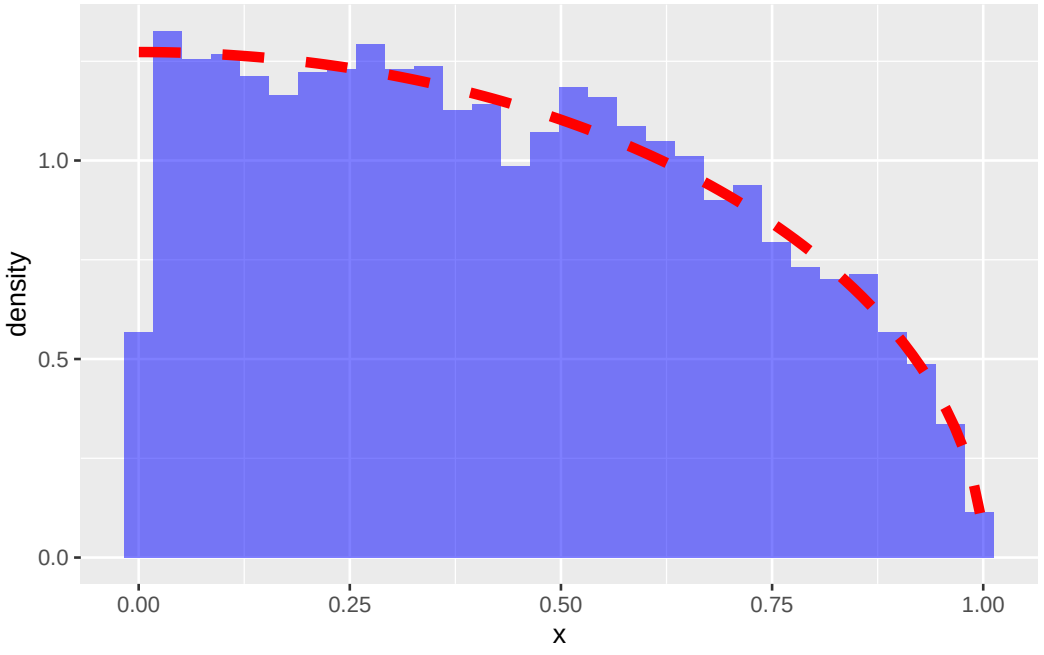
Notice that the following is a valid **probability density function**:

$$f(x) = \frac{4}{\pi}\sqrt{1-x^2}, 0 \leq x \leq 1$$

This curve is just a **scaled version of the unit circle**.


1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00


1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00


density
1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00
x

Interpretation

- We started by drawing U_1 and U_2, and then keeping U_2 if $U_1^2 + U_2^2 \leq 1$.
- This is equivalent to picking U_2, and keeping U_1 if

$$\frac{4}{\pi} U_2 \leq \frac{4}{\pi} \sqrt{1 - U_1^2} = f(U_1)$$

or written another way:

$$U_2 \leq \frac{\pi}{4} \frac{f(U_1)}{g(U_1)}, \quad g(x) = 1 \text{ (pdf of } U_1)$$

- This process generated samples from X (the variable with density $f(x)$)!

Accept-Reject in General

Suppose we want to sample from density (or mass) function:

$$f(x)$$

We know $f(x)$, but can't easily sample from it directly (e.g., Normal distribution).

But what if we had **another density** $g(y)$ such that

$$c \times \frac{g(x)}{f(x)} \geq 1$$

for some $c > 0$ and for all x such that $f(x) > 0$.

In other words we need $cg(x)$ to lie above $f(x)$ for any x where $f(x)$ is positive.

Accept-reject for $f(x)$

For the example we just did the **target density** was

$$f(x) = \frac{4}{\pi}\sqrt{1-x^2}, 0 \leq x \leq 1$$

The **candidate density** was the uniform distribution $g(x) = 1, 0 \leq x \leq 1$

The **constant** was

$$c = \frac{4}{\pi}$$

This ensures that:

$$cg(x) = \frac{4}{\pi} \geq \frac{4}{\pi}\sqrt{1-x^2} = f(x)$$

(NB: c is not unique. We could have picked any $c \geq 4/\pi$.)

Example: Uniform and Beta

Suppose we want to sample from the Beta(2,2) distribution which is:

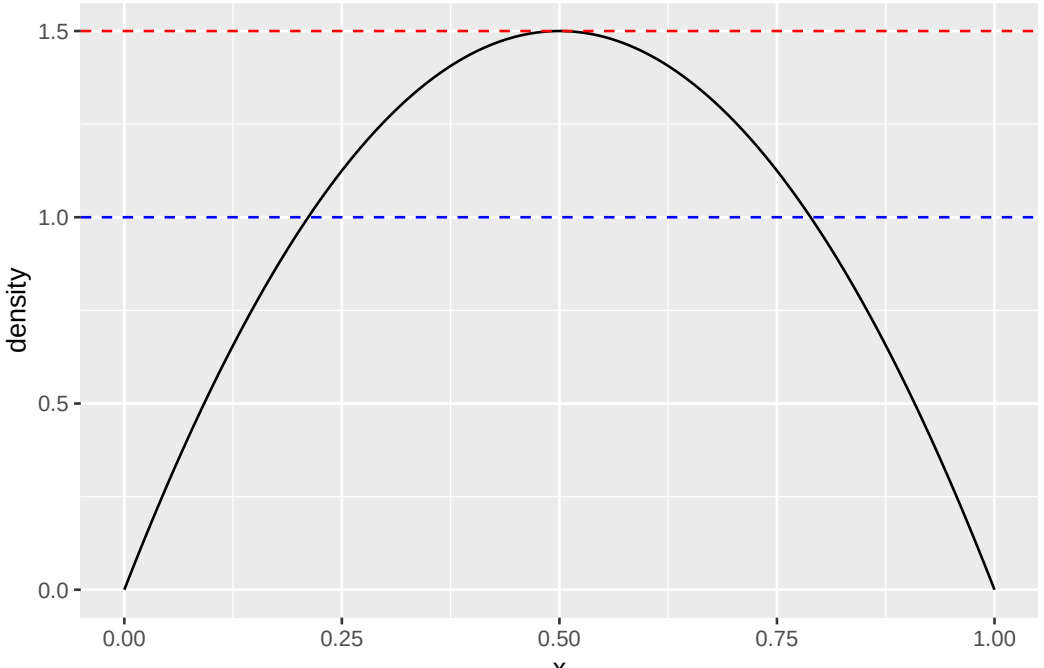
$$f(x) = 6x(1-x), \quad 0 \leq x \leq 1$$

but we only have the standard uniform $g(x) = 1$.

Observe that $f(x)$ achieves its max at $x = 0.5$, the maximum value is $f(0.5) = 6/4$. Therefore we have

$$\frac{6}{4}\frac{g(x)}{f(x)} \geq 1$$

for any point in $[0, 1]$.


1.5
1.0
0.5
0.0
density
0.00
0.25
0.50
0.75
1.00

Accept-Reject Algorithm

- Draw a Y from $g(y)$
- Draw $U \sim U(0,1)$
- If

$$U < \frac{f(Y)}{cg(Y)}$$

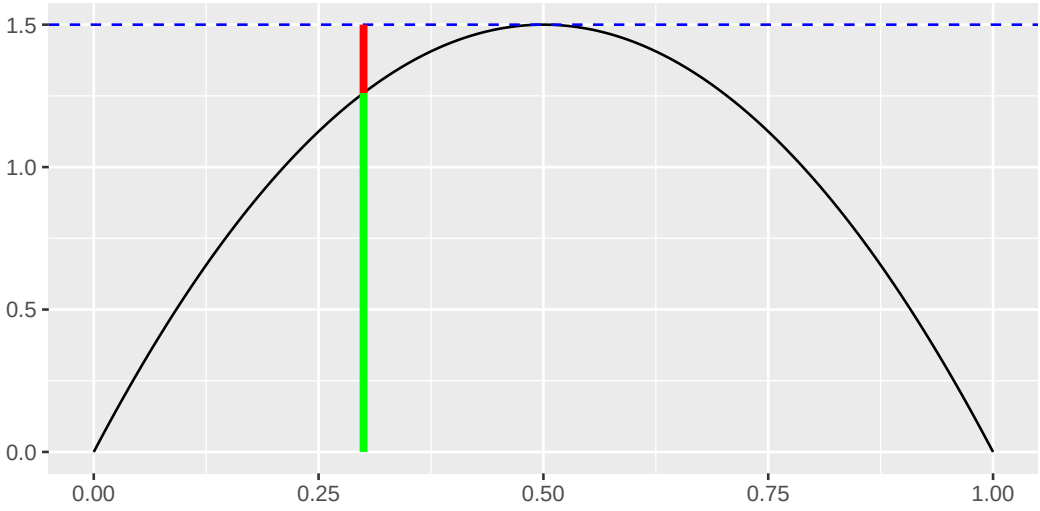
  then **accept** $X = Y$
- Otherwise, **reject** Y as a candidate and repeat.

Example: Beta/Uniform

Say we generate $Y = 0.3$.

- $f(0.3) = 1.26$
- $g(0.3) = 1$
- We will accept if

$$U < \frac{1.26}{(6/4) \times 1} = 0.84$$

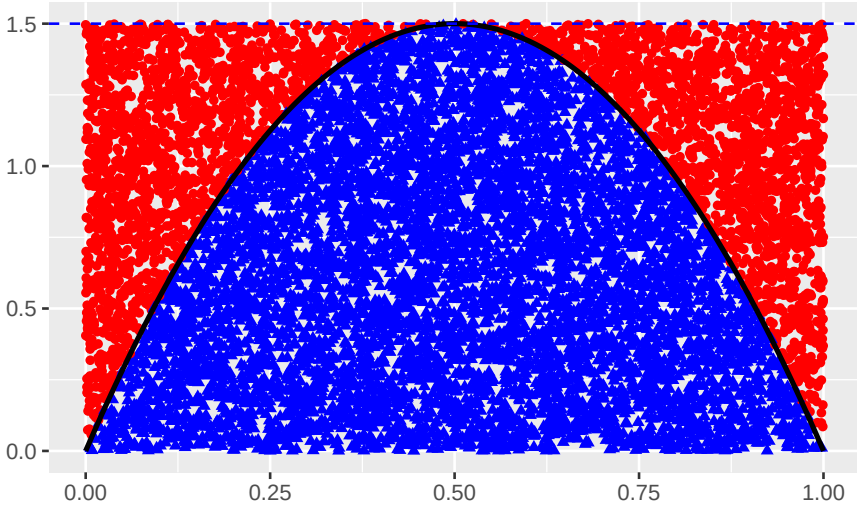

1.5
1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00

R implementation

```
k <- 10000
ys <- runif(k)
fys <- 6 * ys * (1 - ys)
gys <- 1
ratios <- fys / (gys * (6/4))
```

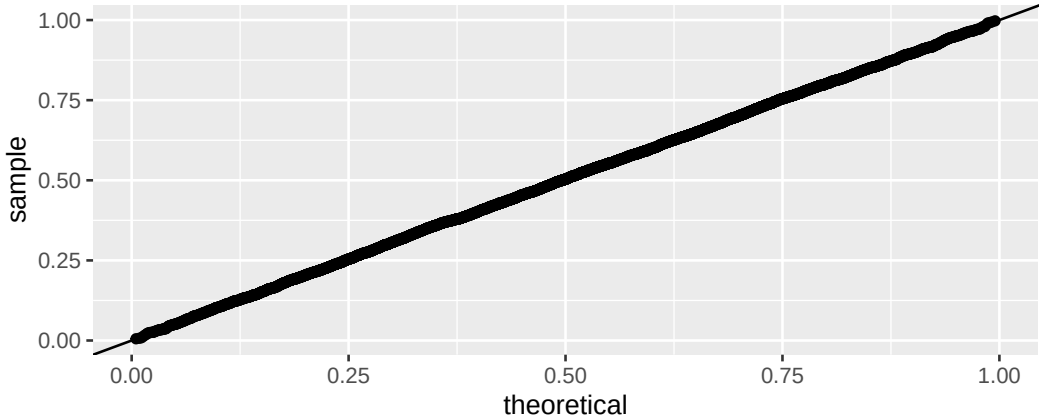
```
us <- runif(k)
accept <- us < ratios
accepted <- ys[accept]
rejected <- ys[!accept] ; mean(!accept)
```

```
## [1] 0.3373
```


1.5
1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00
accept
FALSE
TRUE

Checking the accepted draws

```
ggplot(data.frame(sample = ys[accept]), aes(sample = sample)) +
  geom_qq(distribution = qbeta, dparams = list(shape1 = 2, shape2 = 2)) +
  geom_abline(intercept = 0, slope = 1)
```



Proving the general case

When proving an algorithm generates X we need to show same cumulative distribution function (recall, we did this for inversion method, transformations, etc.).

Let V be the random variable produced by the AR algorithm, we need to show that

$$P(V \leq x) = P(X \leq x)$$

for any x in the support of X.

What kind of variable is V?

We know the distribution of Y and U, but what can we say about V?

Notice that V is set to Y, when $U \leq f(Y)/(cg(Y))$, a conditional distribution:

$$V = Y \,\Big|\, U \leq \frac{f(Y)}{c\, g(Y)}$$

For events A and B, recall the definition of conditional probability:

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

Proof

$$P(V \leq x) = P\left(Y \leq x \,\middle|\, U \leq \frac{f(Y)}{c\,g(Y)}\right) = \frac{P\left(Y \leq x, U \leq \frac{f(Y)}{c\,g(Y)}\right)}{P\left(U \leq \frac{f(Y)}{c\,g(Y)}\right)}$$

We'll take the numerator and denominator separately.

In both cases, think of writing $P(\ldots) = E(I(\ldots))$, e.g.:

$$P\left(Y \leq x, U \leq \frac{f(Y)}{c\,g(Y)}\right) = \mathrm{E}\left(I\left(Y \leq x, U \leq \frac{f(Y)}{c\,g(Y)}\right)\right)$$

Numerator

Notice (a) by independence, the joint density is $1 \times g(y)$ and (b) $I(A, B) = I(A)I(B)$.

$$
\begin{aligned}
P\left(Y \le x, U \le \frac{f(Y)}{c\, g(Y)}\right) &= \int_{-\infty}^{\infty} \int_0^1 I(y \le x) I(u \le f(y)/(cg(y))) g(y)\, du\, dy \\
&= \int_{-\infty}^{\infty} I(y \le x) g(y) \left[\int_0^1 I(u \le f(y)/(cg(y)))\, du\right] dy \\
&= \int_{-\infty}^{x} g(y) \left[\int_0^{f(y)/(cg(y))} 1\, du\right] dy \\
&= \int_{-\infty}^{x} g(y) \frac{f(y)}{cg(y)}\, dy \\
&= \frac{1}{c} \int_{-\infty}^{x} f(y)\, dy = \frac{1}{c} P(X \le x)
\end{aligned}
$$

Denominator

$$P\left(U \leq \frac{f(Y)}{c\, g(Y)}\right) = \mathrm{E}\left(I\left(U \leq \frac{f(Y)}{c\, g(Y)}\right)\right)$$

By similar logic,

$$\begin{aligned} P\left(U \leq \frac{f(Y)}{c\, g(Y)}\right) &= \int_{-\infty}^{\infty} g(y) \left[\int_0^{f(y)/(cg(y))} 1\, du\right] dy \\ &= \int_{-\infty}^{\infty} g(y) \frac{f(y)}{cg(y)}\, dy \\ &= \int_{-\infty}^{\infty} \frac{1}{c} f(y)\, dy = \frac{1}{c} \end{aligned}$$

Taking the ratio, the $(1/c)$ cancels, so $P(V \leq x) = P(X \leq x)$.

Example: Truncated Normal

We've used the **rejection sampling algorithm** for the Truncated Normal before:

$$Y = Z \mid 0 \leq Z \leq 1, Z \sim N(0, 1)$$

To use AR, we need the **density function**. We found the CDF earlier:

$$F_Y(y) = \frac{\Phi(y) - \Phi(0)}{\Phi(1) - \Phi(0)}, \quad 0 \leq y \leq 1$$

The PDF is its derivative. With ϕ the standard normal PDF, for $0 \leq y \leq 1$,

$$f(y) = \frac{\phi(y)}{\Phi(1) - \Phi(0)}$$

Implementing in R

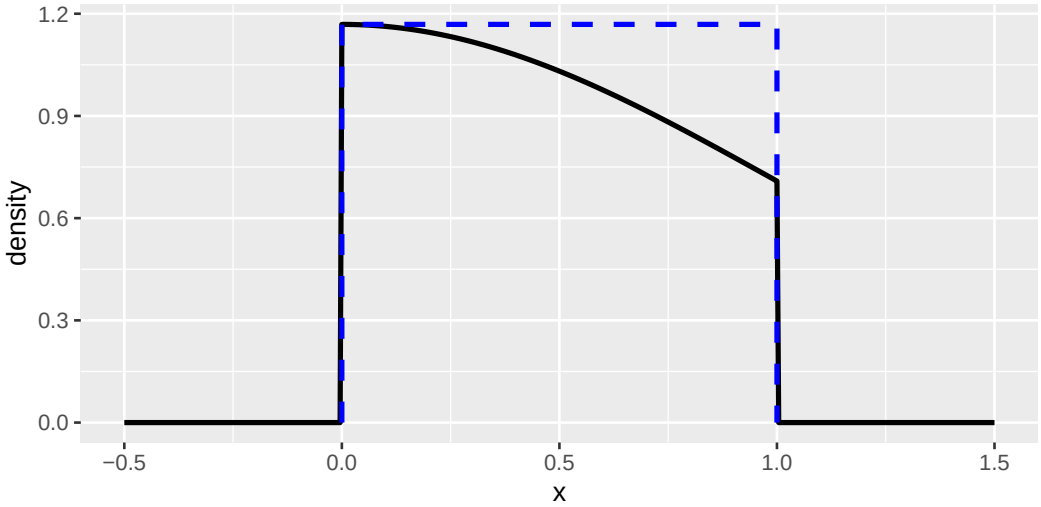
```
(scaling_const <- pnorm(1) - pnorm(0))
```

```
## [1] 0.3413
```

Then we compute the density function as:

```
truncated <- function(x) {
  ifelse(x >= 0 & x <= 1,
         dnorm(x) / scaling_const,
         0)
}
```

Picking a candidate density: Uniform



Implementing

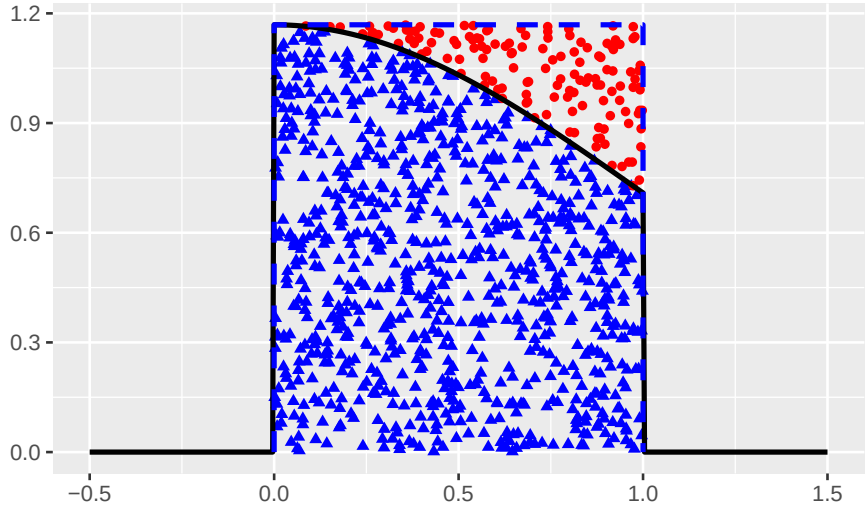
```
ys <- runif(1000)
const <- truncated(0)
ratios <- truncated(ys) / (const * 1) # g(y) = 1
```

```
us <- runif(1000)
accept_uniform <- us < ratios
accepted_uniform <- ys[accept_uniform]
rejected_uniform <- ys[!accept_uniform]
```

Estimating $\mathsf{E}(X)$:

```
mean(accepted_uniform)
```

```
## [1] 0.4619
```



We rejected 13.8% of the candidate draws. Can we do better?

Fitting a closer candidate distribution

Consider the density:

$$g(y) = \frac{2}{3}(2 - y), \quad 0 \leq y \leq 1$$

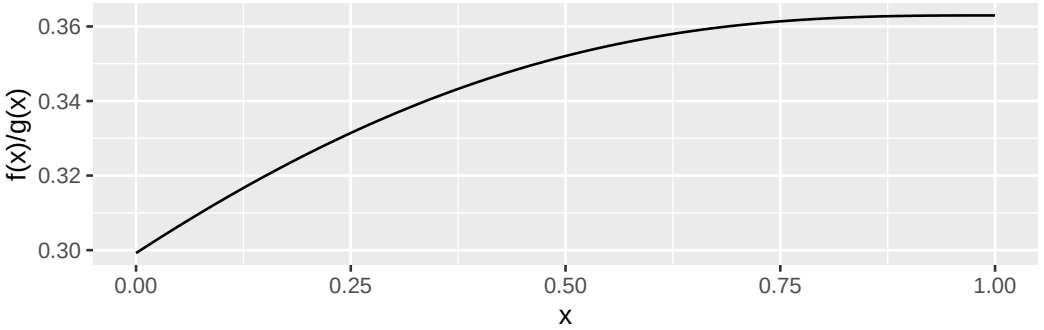
with quantile function

$$Q_y(p) = 2 - \sqrt{4 - 3p}$$

Need to find a c such that

$$c \times \frac{g(x)}{f(x)} \geq 1 \Rightarrow c \geq \frac{f(x)}{g(x)}, 0 \leq x \leq 1$$

Notice that the **ratio** $f(x)/g(x)$ **is increasing**:

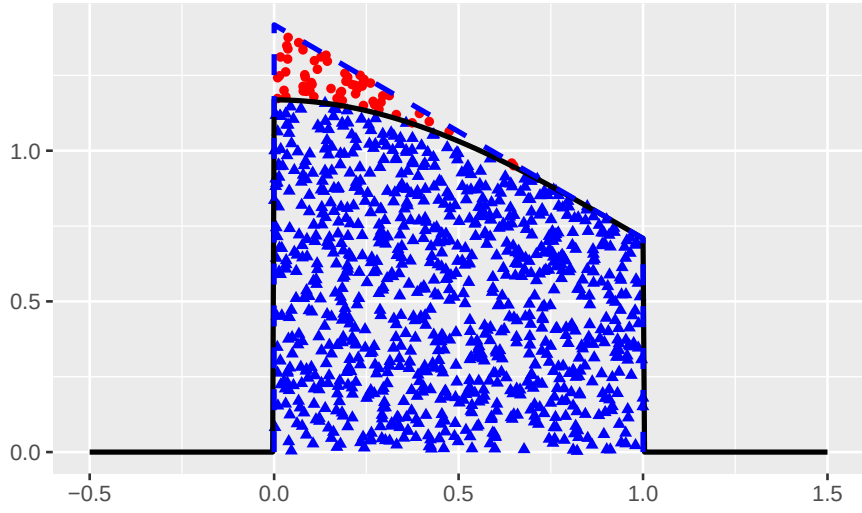


Then the maximum ratio is at $x = 1$.

$$c = f(1)/g(1) = 1.063 \Rightarrow \frac{cg(x)}{f(x)} \geq 1, 0 \leq x \leq 1$$

```
g <- function(y) { (2/3) * (2 - y)}
qg <- function(p) { 2 - sqrt(4 - 3 * p)}
ys <- qg(runif(1000))
const <- truncated(1) / g(1)
ratios <- truncated(ys) / (const * g(ys))
```

```
us <- runif(1000)
accept_g <- us < ratios
accepted_g <- ys[accept_g]
rejected_g <- ys[!accept_g]
```



Now we reject only 6.3% of the candidate draws.

Comparing Efficiency

Recall: **more efficient** means lower variance. Both envelopes give exact draws from f, and from m candidates, accept-reject keeps about $N = m/c$, so the mean of the kept draws has

$$\operatorname{Var}\left(\bar{X}\right) = \frac{\operatorname{Var}(X)}{N} \approx \frac{c \operatorname{Var}(X)}{m}.$$

So for a fixed m, the ratio of the variances is the ratio of the constants:

```
c_uniform <- truncated(0)
c_g <- truncated(1) / g(1)
(rel_eff <- c_uniform / c_g)
```

```
## [1] 1.099
```

The uniform envelope's estimate has 10% more variance than the estimate using g.

A bimodal density

Suppose we have a density of the form:

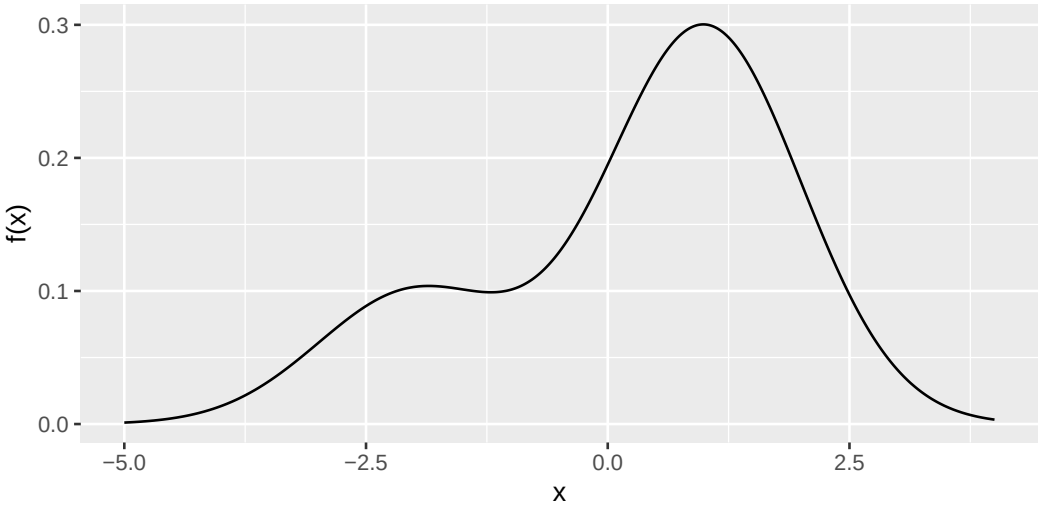
$$f(x) = 0.25\phi(x+2) + 0.75\phi(x-1), -\infty < x < \infty$$

where ϕ is the standard Normal PDF.

```
f <- function(x) { 0.25 * dnorm(x + 2) + 0.75 * dnorm(x - 1)}
```

A density like this is called **bimodal** because it contains two local maxima.

Density of f



What candidate distribution?

Our initial thought might be to use $\phi(x)$ (standard Normal) as the candidate.

The only problem with that is that

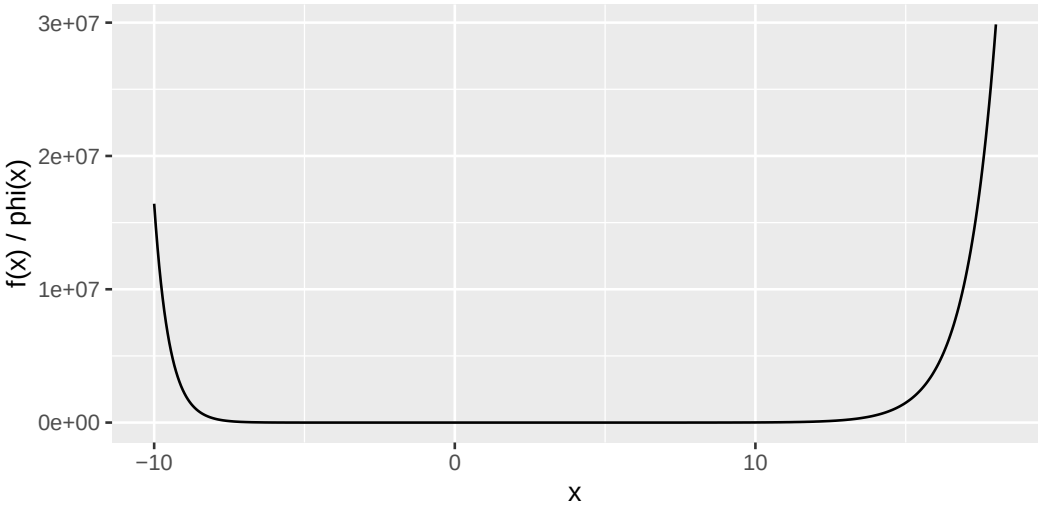
$$\lim_{x \to \pm\infty} f(x)/\phi(x) = \infty$$

(i.e., no c exists that uniformly bounds f).

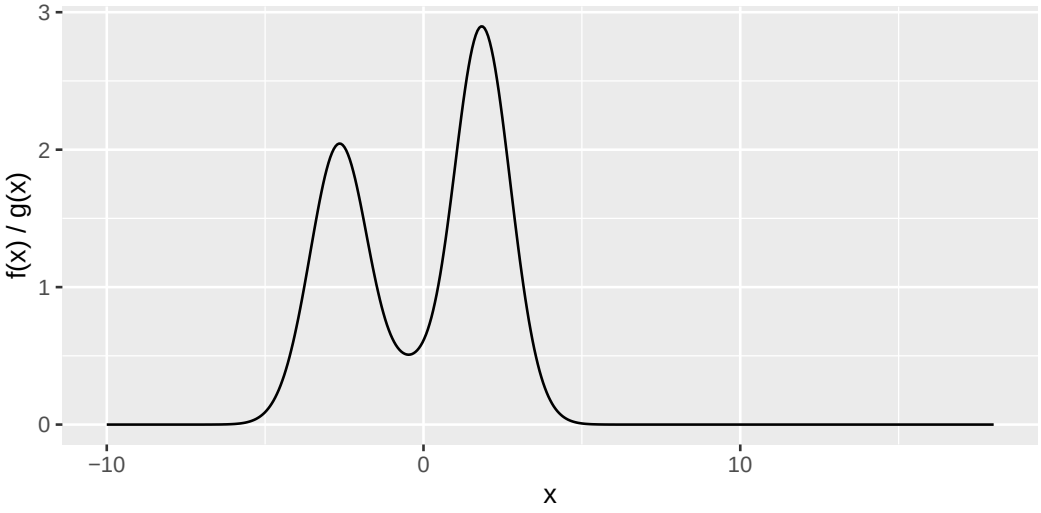
We saw that the Cauchy distribution has **fat tails**, perhaps that would be useful?

$$g(x) = \frac{1}{\pi(1 + x^2)}$$

Ratio of $f(x)/\phi(x)$



Ratio of $f(x)/g(x)$

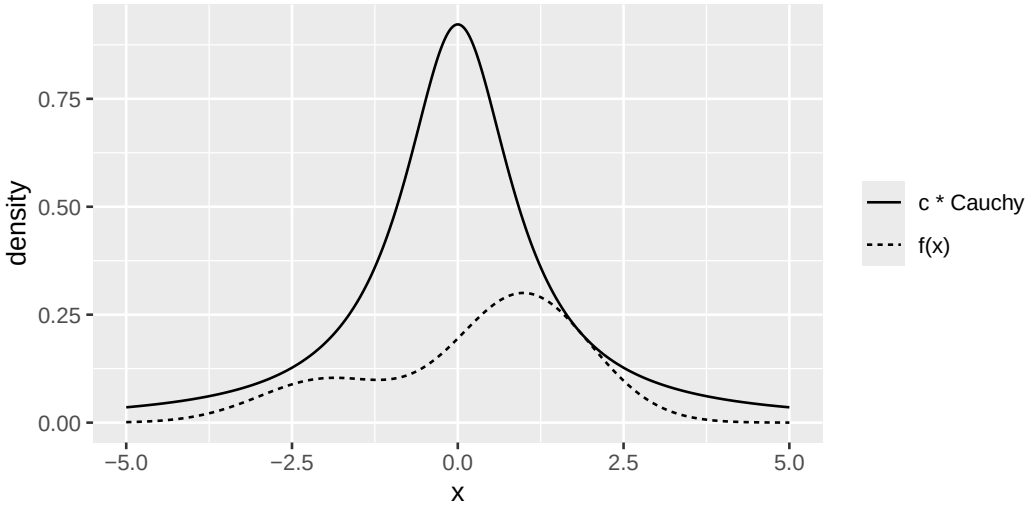


Lazy mode: find c by evaluation

```
h <- function(x) { f(x) / dcauchy(x)}
xs <- seq(-4, 4, length.out = 1000)
(const <- max(h(xs)))
```

```
## [1] 2.898
```

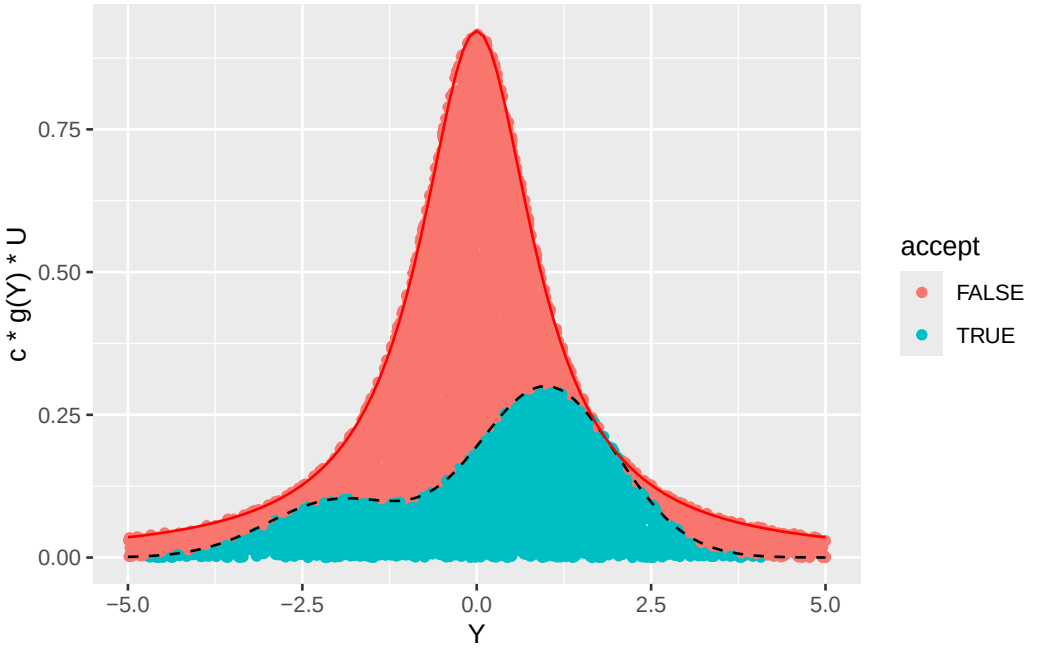
Plotting Distributions



Putting it together

```
k <- 10000
ys <- rcauchy(k)
ratios <- f(ys) / (const * dcauchy(ys))
us <- runif(k)
accept <- us < ratios
x <- ys[accept] ; mean(accept)
```

```
## [1] 0.3496
```


0.75
0.50
0.25
0.00
c * g(Y) * U
−5.0
−2.5
0.0
2.5
5.0
Y
accept
FALSE
TRUE

What if we got c wrong?

We found $c = 2.898$. What if we were wrong? How would the distribution change? Too large, too small?

```
wrong_consts <- c(2 *  const,
                  const,
                  0.75 * const,
                  0.25 * const,
                  0.1 * const,
                  0)
```

Functions to handle AR

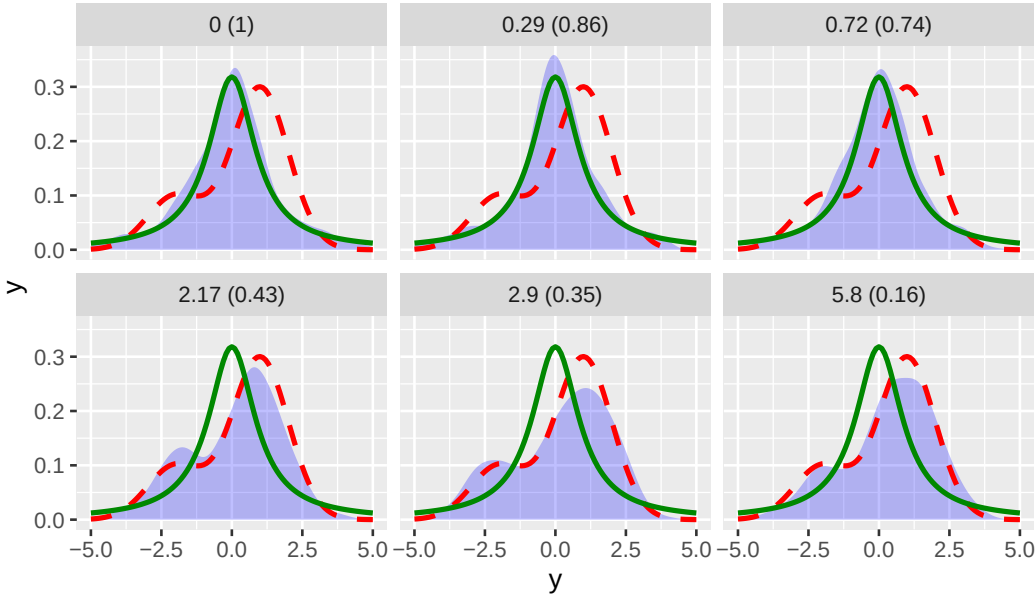
For numerical reasons, we'll rewrite $U \leq f(Y)/(cg(Y))$ as

$$c\, g(Y)\, U \leq f(Y)$$

```
accept_reject <- function(n, f, g, rg, const) {
    ys <- rg(n)
    us <- runif(n)
    accept <- us * const * g(ys) <= f(ys)
    data.frame(y = ys, accept = accept)
}
```

Using the bad constants

```
bimodal <- function(const) {
    accept_reject(1000, f, dcauchy, rcauchy, const)
}
wrong <- map_dfr(wrong_consts, .id = "constant", bimodal)
wrong$constant <- wrong_consts[as.numeric(wrong$constant)] # get the origi
```


0 (1)
0.29 (0.86)
0.72 (0.74)
2.17 (0.43)
2.9 (0.35)
5.8 (0.16)
0.0
0.1
0.2
0.3
y
−5.0
−2.5
0.0
2.5
5.0
y

Accept-Reject Summary

- For a density f, use a density g for the sampling.
- Draw uniform values to decide to accept or reject draws from g.
- Need to figure out the scaling constant c to make $cg(y)/f(y) \geq 1$
- Generally, we accept about $100 \times (1/c)\%$ of the candidates, so making c small is useful.
- Picking c too large is ok, too small leads to draws from g.